

WezTerm, Neovim, and Herdr Tutorial

This tutorial explains how to use the terminal environment installed by this repository. It assumes that you are new to terminal multiplexers and modal editors.

The tool is named Herdr, not Herder. The shell alias `h` starts it.

This tutorial assumes the repository has remained in the same location where `./install.sh --apply` was run. The active configuration consists of Stow links back into that clone. Moving, renaming, or deleting the repository afterward breaks those links; choose a permanent location before applying the setup.

1. The mental model

The three tools have different jobs:

```
WezTerm window
├─ Herdr workspace
├─ pane: Claude Code or Codex
├─ pane: Neovim
└─ pane: tests, server, or ordinary shell
```

- WezTerm is the macOS terminal window. It renders text, accepts keyboard input, provides tabs, and shows the top system-status bar.

- Herdr organizes persistent workspaces, tabs, and panes inside the terminal. It is especially useful when coding agents run for a long time.

- Neovim edits files and reviews changes made by you or an agent.

This setup does not enable WezTerm's multiplexer. Use Herdr for agent-heavy work, or tmux for general and remote work, instead of nesting all three.

2. Start in WezTerm

Open WezTerm from Spotlight or the Applications folder. A plain zsh shell starts automatically.

The top bar contains WezTerm tabs on the left and system information on the right:

```
[1. zsh] CPU 12% MEM 62% | BAT+ 72% | Tue Sep 15 14:30
```

- CPU is total CPU activity normalized across the logical CPU cores.
- MEM is non-cache memory use: anonymous application memory, wired memory, and physical pages occupied by compressed memory. It excludes file-backed cache that macOS can reclaim for applications, so it is more useful than simply subtracting free pages from total RAM.

- BAT+ means the battery is charging; BAT means it is not charging.

- The display refreshes every ten seconds.

Useful WezTerm shortcuts on macOS

Action	Shortcut
New WezTerm tab	<code>Command-t</code>
Close current tab	<code>Command-w</code>

Action	Shortcut
Previous or next tab	<code>Command-Shift-[/ Command-Shift-]</code>
Select tab 1-9	<code>Command-1 … Command-9</code>
New WezTerm window	<code>Command-n</code>
Previous or next window	<code>Command-\ / Command-Shift-\</code>
Select window 1-9	<code>Command-Option-1 … Command-Option-9</code>
Move active pane to a new window	<code>Command-Shift-m</code>
Scroll one page	<code>Command-Up / Command-Down</code>
Scroll three lines	<code>Command-Shift-Up / Command-Shift-Down</code>
Copy / paste	<code>Command-c / Command-v</code>
Search terminal output	<code>Command-f</code>
Increase / decrease font	<code>Command++ / Command--</code>
Reset font size	<code>Command-0</code>
Reload WezTerm configuration	<code>Command-r</code>

The window title shows its current number, such as [1] or [2]. Window numbers match the `Command-Option-1` through `Command-Option-9` shortcuts. A WezTerm tab is a tab inside one window; `Command-Shift-m` takes the active Herdr or shell pane out into a separate window without requiring an interactive shell command.

Use WezTerm tabs for separate top-level activities. Use Herdr's tabs and panes for the related processes within one project.

3. Shell conveniences

The shell defines a few short commands:

Command	Meaning
<code>h</code>	Start or attach to Herdr
<code>v</code>	Start Neovim
<code>t</code>	Start or attach to tmux
<code>ll</code>	Detailed directory listing with Git information
<code>tree</code>	Show a directory tree

`ls`, `ll`, and `tree` emit terminal hyperlinks for displayed paths. Hold `Command` and click a URL to open it in the browser, or click a `file://` path to open the local file with its macOS default application. This remains available when Codex, Neovim, or another pane application is using mouse reporting because the configured Command modifier is handled by WezTerm.

Other interactive features include:

- autosuggestions based on command history;
- syntax coloring that warns about invalid commands before execution;
- extra command completions;
- Atuin history search;
- fzf fuzzy selection;
- zoxide directory navigation;
- the Starship prompt.

When a faint autosuggestion appears, press the right-arrow key to accept it.

Use `Ctrl-r` to search command history. After zoxide learns your directories, `z project-name` can replace a long `cd` command.

Intentional tools in this setup

These tools are installed by the repository because they support the workflow; they are not random extras. `./install.sh --apply --packages` installs the Homebrew entries in `Brewfile`. Claude Code and Codex are the exception: they are installed separately with `./scripts/agents.sh` because their publishers provide native installers.

Tool	What it is for	Start here
Atuin	Search and reuse shell history	<code>↑</code> or <code>Ctrl-r</code> ; type a query, <code>Enter</code> edits the selected command, <code>Esc</code> exits
fzf	Fuzzy selection used by shell integrations	<code>Ctrl-t</code> files, <code>Alt-c</code> directories, or use it through picker commands
zoxide	Jump to frequently used directories	<code>z project-name</code>
Starship	Prompt showing Git and runtime context	It starts automatically with <code>zsh</code>
zsh plugins	Suggestions, syntax highlighting, and completions	Type normally; accept a suggestion with <code>Right Arrow</code>
bat	Read files with syntax highlighting	<code>cat README.md</code>
eza	Modern directory listings and trees	<code>ls</code> , <code>ll</code> , <code>tree</code> ; <code>⌘</code> -click printed paths
ripgrep / fd	Search text / find files quickly	<code>rg "pattern" ./ fd filename</code>
jq	Inspect and transform JSON	<code>jq '.items[]' data.json</code>
delta	Readable Git diffs	<code>git diff</code>
mise	Manage Node 24 and pnpm 10.28.0	<code>mise current</code> , <code>mise install</code>
uv	Manage Python project environments	<code>uv sync</code> or <code>uv run ...</code> inside a Python project
GitHub CLI	Work with GitHub from the terminal	<code>gh auth status</code> , <code>gh pr list</code>

Atuin’s Up-arrow screen is intentional: it replaces the usual single-command history step with a searchable history list. Use `↑/↓` to select a result, `Enter` or `Tab` to edit it at the prompt, `Ctrl-o` to inspect it, and `Esc` to leave without selecting anything. The command is not executed until you submit it from the normal shell prompt.

4. Learn Neovim’s modes first

Start Neovim in a project:

```
cd /path/to/project
v .
```

Neovim is a modal editor. A key performs different actions depending on the current mode.

Mode	Purpose	Enter it	Leave it
Normal	Navigate and run commands	Neovim starts here	-
Insert	Type text	<code>i</code> , <code>a</code> , or <code>o</code>	<code>Esc</code>
Visual	Select text	<code>v</code> or <code>V</code>	<code>Esc</code>

Mode	Purpose	Enter it	Leave it
Command	Save, quit, and run editor commands	:	Enter or Esc

If you are unsure which mode you are in, press `Esc`. This returns you to Normal mode in most situations.

Your first edit

1. Press `Space Space` to open the smart file picker.
2. Type part of a filename.
3. Use the arrow keys to select it and press `Enter`.
4. Move with `h`, `j`, `k`, and `l`, or use the arrow keys initially.
5. Press `i`, type a change, and press `Esc`.
6. Press `Space w` to save.
7. Type `:q` and press `Enter` to quit.

Common exit commands:

Command	Meaning
<code>:w</code>	Save
<code>:q</code>	Quit if there are no unsaved changes
<code>:wq</code>	Save and quit
<code>:q!</code>	Discard unsaved edits and quit

Basic Normal-mode movement and editing

Action	Key
Move left/down/up/right	<code>h / j / k / l</code>
Next / previous word	<code>w / b</code>
Start / end of line	<code>0 / \$</code>
Start / end of file	<code>gg / G</code>
Undo / redo	<code>u / Ctrl-r</code>
Delete current line	<code>dd</code>
Copy current line	<code>yy</code>
Paste after cursor	<code>p</code>
Search in file	<code>/text</code> , then <code>Enter</code>
Next / previous match	<code>n / N</code>

The system clipboard is enabled. Text copied in Neovim can be pasted into other macOS applications, and vice versa.

5. Use this Neovim configuration

In the tables below, `Space` is the leader key. For example, `Space g g` means press `Space`, release it, then press `g` twice.

Find code

Action	Shortcut
Smart file picker	<code>Space Space</code>
Search text across the project	<code>Space /</code>
Browse Git branches	<code>Space g b</code>

Action	Shortcut
Browse Git history	Space g l

The picker supports fuzzy search. Type only distinctive parts of a filename or phrase, select a result, and press Enter. Press Esc to close it.

Work with editor splits

Create splits with `:vsplit` or `:split`, then move between them with:

Direction	Shortcut
Left	Ctrl-h
Down	Ctrl-j
Up	Ctrl-k
Right	Ctrl-l

These are Neovim splits inside one terminal pane. They are different from Herdr panes.

Review Git changes with Gitsigns

Changed lines have signs in the left column. The current line also shows its Git blame after a short delay.

Action	Shortcut
Next / previous changed hunk	]h / [h
Preview the current hunk	Space h p
Stage the current hunk	Space h s
Reset the current hunk	Space h r
Toggle current-line blame	Space h b

Be careful with `Space h r`: it discards the current uncommitted hunk.

Review the whole repository with Neogit

Press `Space g g` to open Neogit. It provides a repository-wide Git view for reviewing files, diffs, staging, and commits. Press `?` inside Neogit to see its context-sensitive controls; use those displayed controls instead of trying to memorize everything immediately.

Which-key also shows available leader-key actions when you pause after pressing Space.

6. Start a Herdr workspace

Start with a real Git project:

```
cd /path/to/project
h
```

Running `h` is the same as running `herdr`. It launches or attaches to the default persistent Herdr session and creates or opens a workspace.

Learn Herdr's hierarchy in this order:

```

session
├─ workspace: one project or task
├─ tab: one layout, such as "development"
├─ pane: one shell, editor, agent, test process, or server

```

- A session is the background Herdr server. Most people need only the default session.

- A workspace normally represents one repository or task.
- A tab groups a pane layout, such as agents, tests, or server.
- A pane contains one real terminal process.

Start with the mouse: click a pane or tab to focus it, drag pane borders to resize, and right-click to open contextual menus.

7. Understand the Herdr prefix

Herdr commands use `Ctrl-b` as a prefix. This is a sequence, not one large keyboard chord.

For `Ctrl-b`, then `v`:

1. Hold `Ctrl` and press `b`.
2. Release both keys.
3. Press `v`.

The first step tells Herdr that the next key is a Herdr command instead of input for the focused shell or editor.

Press `Ctrl-b`, then `?` at any time to show the active keybindings.

Essential Herdr keys

Action	Sequence
Show help	<code>Ctrl-b</code> , then <code>?</code>
Split with a new pane on the right	<code>Ctrl-b</code> , then <code>v</code>
Split with a new pane below	<code>Ctrl-b</code> , then <code>-</code>
Focus left/down/up/right pane	<code>Ctrl-b</code> , then <code>h/j/k/l</code>
Cycle to next pane	<code>Ctrl-b</code> , then <code>Tab</code>
Close pane	<code>Ctrl-b</code> , then <code>x</code>
Zoom/unzoom pane	<code>Ctrl-b</code> , then <code>z</code>
Enter resize mode	<code>Ctrl-b</code> , then <code>r</code>
Toggle sidebar	<code>Ctrl-b</code> , then <code>b</code>
Detach from Herdr	<code>Ctrl-b</code> , then <code>q</code>

Tabs and workspaces

Action	Sequence
New tab	<code>Ctrl-b</code> , then <code>c</code>
Previous / next tab	<code>Ctrl-b</code> , then <code>p/n</code>
Select tab 1-9	<code>Ctrl-b</code> , then <code>1 ... 9</code>
Rename tab	<code>Ctrl-b</code> , then <code>Shift-t</code>
Close tab	<code>Ctrl-b</code> , then <code>Shift-x</code>
Open workspace picker	<code>Ctrl-b</code> , then <code>w</code>
Create workspace	<code>Ctrl-b</code> , then <code>Shift-n</code>

Action	Sequence
Rename workspace	Ctrl-b, then Shift-w
Create a Git-worktree workspace	Ctrl-b, then Shift-g

Uppercase actions mean holding Shift for the action key after releasing the prefix.

8. A practical agent workflow

Open a project and start Herdr:

```
cd ~/Documents/Projects/example-project
h
```

Use the first pane for an agent:

```
claude
```

or:

```
codex
```

Then build this layout:

1. Press Ctrl-b, then v to create a pane on the right.
2. In the new pane, run v . to open Neovim.
3. Press Ctrl-b, then - to create a pane below.
4. In the bottom pane, run the project's tests or development server.

The result is:

```

┌ Claude Code / Codex | Neovim |
├ | review and edit |
└
┌ tests, server, logs, or shell |
└
```

When the agent changes files:

1. Focus Neovim with Ctrl-b, then l or by clicking it.
2. Use]h and [h to visit changed hunks.
3. Use Space h p to inspect a hunk.
4. Use Space g g for the repository-wide review.
5. Run relevant tests in the bottom pane.
6. Return to the agent pane and give feedback or the next task.

9. Detach, reattach, and stop

Detach without stopping the panes:

```
Ctrl-b, then q
```

You can also close the WezTerm window. The Herdr background server and its pane processes continue running.

Later, open WezTerm and reattach:

```
h
```

To inspect Herdr outside its interface:

```
herdr status
herdr workspace list
herdr session list
```

To actually stop the default Herdr server and its running environment:

```
herdr server stop
```

Do not use `server stop` merely to leave the interface; detach instead.

10. Agent completion notifications

Herdr is the agent-aware notification source, while WezTerm delivers the macOS notification. The configured flow is:

```
Claude Code or Codex changes state
↓
Herdr recognizes done or needs-attention
↓
Herdr emits a terminal notification
↓
WezTerm asks macOS to display it
```

Notifications are forwarded even while the same WezTerm window is focused so agent completion and attention events are not silently discarded. Herdr sound also remains enabled for agent changes in background workspaces.

WezTerm can initially request provisional macOS notification authorization. macOS does not show a permission dialog for provisional authorization and may place the first notification quietly in Notification Center. Test the complete route, then look in Notification Center if no banner appears:

```
herdr notification show "myagentterminal" --body "Notifications are working" --sound done
```

The repository enables notifications even while WezTerm is focused with:

```
config.notification_handling = "AlwaysShow"
```

Reload WezTerm with `Command-r` after changing this setting manually.

This path is more precise than reacting to every terminal bell: Herdr knows which supported agent changed state, whereas WezTerm alone only knows that a program emitted a notification or bell sequence.

11. Optional agent integrations

Herdr can detect supported agents from their terminal output. Optional native integrations can provide richer lifecycle state or session restoration.

Check their current status:

```
herdr integration status
```

Install only the integrations you want:

```
herdr integration install claude
herdr integration install codex
```

These integrations are separate from `AGENTS.md`, but they do write hook files inside the agents' configuration directories. Treat them as an explicit, optional setup step rather than part of the automatic `myagentterminal` installation.

12. Troubleshooting

A Herdr shortcut does nothing

Press `Ctrl-b`, release it, and only then press the action key. Use `Ctrl-b`, then `?` to verify the current bindings. The macOS desktop or outer terminal can intercept some direct key combinations, but the configured prefix sequences avoid most conflicts.

You accidentally opened Herdr inside Herdr

Nested Herdr sessions are disabled. If `$HERDR_ENV` is 1, you are already in a Herdr pane:

```
echo $HERDR_ENV
```

Use the existing workspace rather than running `h` again.

An agent is not detected

Run:

```
herdr agent list
herdr integration status
```

Herdr logs are under `~/.config/herdr/`. Because that directory is linked to the `myagentterminal` repository, the corresponding `*.log` files and named-session runtime directory are explicitly ignored by Git; only `config.toml` should be tracked.

Agent completion notifications do not appear

Confirm that WezTerm notifications are allowed in macOS System Settings. Reload the WezTerm configuration with `Command-r`, reload Herdr with `herdr server reload-config`, and run the notification test from section 10.

The configured `AlwaysShow` mode also forwards notifications while the WezTerm window is focused. On first use, provisional authorization may place the alert quietly in Notification Center instead of showing a banner.

Neovim seems stuck

Press `Esc`, then decide whether to save:

```
:wq save and quit
:q! discard changes and quit
```

The WezTerm status bar does not update

Press `Command-r` to reload the configuration. If it still does not appear, restart WezTerm and validate the configuration with:

```
wezterm --config-file ~/.config/wezterm/wezterm.lua ls-fonts >/dev/null
```

If a URL or local file does not open, confirm that the path was printed by

`ll` or `tree`, hold `Command` while clicking, and reload WezTerm with `Command-r`. For a `file://` path, macOS decides which default application opens the file.

13. A small practice exercise

Use a disposable Git repository:

```
mkdir -p /tmp/myagentterminal-practice
cd /tmp/myagentterminal-practice
git init
printf '# Practice\n' > README.md
h
```

Then:

1. Open Neovim with `v ..`
2. Find `README.md` with `Space Space`.
3. Insert a new line and save it with `Space w`.
4. Preview the changed hunk with `Space h p`.
5. Open Neogit with `Space g g`.
6. Create a Herdr split with `Ctrl-b`, then `v`.
7. Run `git diff` in the new pane.
8. Detach with `Ctrl-b`, then `q`.
9. Reattach with `h` and confirm that the panes survived.

Once this sequence feels comfortable, use the same layout in a real project.

Official references

- WezTerm: [⟨https://wezterm.org/config/files.html⟩](https://wezterm.org/config/files.html)
- Neovim user manual: [⟨https://neovim.io/doc/user/⟩](https://neovim.io/doc/user/)
- Herdr concepts: [⟨https://herdr.dev/docs/concepts/⟩](https://herdr.dev/docs/concepts/)
- Herdr keyboard guide: [⟨https://herdr.dev/docs/keyboard/⟩](https://herdr.dev/docs/keyboard/)
- Herdr configuration: [⟨https://herdr.dev/docs/configuration/⟩](https://herdr.dev/docs/configuration/)